HavenMade – Project Q&A and Design Defense

Haven Freeman

CSC 4370 – Web Programming

Project 1: HavenMade – Recipe Studio

Q1 – Purpose & Audience

Question: What problem or need does your chosen topic address, and who is its primary audience?

I created HavenMade because I wanted a recipe website that felt simple, cozy, and easy to use. A lot of recipe websites have so much information that it can be hard to find what you actually need. I wanted mine to focus on affordable meals and recipes that aren't too complicated.

My main audience is college students, beginner cooks, and people who want good homemade food without spending a lot of money. I also wanted the website to feel welcoming instead of overwhelming.

Code reference: In index.html, the homepage introduces HavenMade with the tagline "Good food. Small budgets. A little peace in every recipe." The featured recipes and buttons help visitors start exploring.

Q2 – Topic & Extension

Question: How does the extension support the topic's story or primary user flow?

My extra feature is the recipe recommendation section on the Recipes page. I added it because sometimes the hardest part of cooking is figuring out what you want to make.

Visitors can choose between four options: On a Budget, Something Quick, Comfort Food, and Something Sweet. Based on what they select, the website suggests a recipe.

I think this fits HavenMade because the goal of my website is to make cooking feel easier and less stressful.

Code reference: In recipes.html, the recommendation section starts with <section class="section-container recommendation-section">. The four options use radio buttons with IDs mood-budget, mood-quick, mood-cozy, and mood-sweet.

Q3 – Navigation & Page Flow

Question: Walk through a visitor's path from the home page to a key detail page.

When someone opens HavenMade, they start on the homepage. They can click Explore Recipes or use the navigation menu to go to the Recipes page.

On the Recipes page, they can browse six different recipes. When they click one, it takes them to the recipe detail page, where they can see the ingredients and cooking instructions.

For example, clicking Chicken Noodle Soup takes them to its section on the detail page. I also included a My Story page so visitors can learn more about why I created HavenMade.

Code reference: In recipes.html, the Chicken Noodle Soup card links to recipe-detail.html#chicken-soup. In recipe-detail.html, the matching recipe section has id="chicken-soup". This allows the link to take visitors directly to that recipe.

Q4 – Semantic HTML & Content

Question: Show where semantic landmarks clarify the structure of a page.

I tried to organize my HTML so that each part of the website has a clear purpose. Instead of putting everything into random containers, I used elements like header, nav, main, section, article, and footer.

For example, I used article for individual recipe cards because each card represents its own piece of content. I also used headings to separate the different parts of the pages.

This makes the HTML easier to understand and gives the website a more organized structure.

Code reference: In index.html, the navigation uses <nav class="navbar" aria-label="Main navigation">. The featured recipes use <article class="recipe-card">. In recipe-detail.html, each full recipe is also inside an article.

Q5 – Visual Design System

Question: How do your typography, palette, and spacing reinforce the topic?

I wanted HavenMade to feel like a cozy, slightly retro recipe website. I chose cream, chocolate brown, pink, yellow, and sage green because those colors remind me of a warm kitchen.

I also used rounded recipe cards and soft-looking fonts because I didn't want the website to feel too serious or plain. I wanted it to have personality while still being easy to read.

I kept the design consistent across all four pages so they would feel like parts of the same website.

Code reference: At the top of css/style.css, I created CSS variables for the colors, including --cream, --butter, --pink, --sage, and --cocoa. I used the Google Fonts **DM Sans** for regular text and **Fredoka** for headings and the logo.

Q6 – Layout & Responsiveness

Question: Show a layout changing between desktop and mobile widths.

I wanted HavenMade to work on both computers and phones, so I used responsive CSS.

On larger screens, the recipe cards can appear in three columns. On smaller screens, they change to two columns and then one column so the pictures and text don't become too small.

The homepage also changes its layout on smaller screens, and the navigation adjusts to fit better.

Code reference: In css/style.css, .recipe-grid uses grid-template-columns: repeat(3, 1fr). The media query at max-width: 900px changes it to two columns, and the one at max-width: 600px changes it to one column. The .hero section also changes from two columns to one.

Q7 – Motion & CSS Interaction

Question: How does a CSS-only reveal, hover, focus, or checked state work in your site?

One of my favorite interactive parts is the recipe recommendation feature. I used HTML radio buttons and CSS so that choosing a different mood changes the recipe being displayed.

I also added animations to make the website feel more lively. For example, the homepage image has a gentle floating effect, and some page elements fade into view.

I wanted the animations to make the website feel nicer without being distracting.

Code reference: In css/style.css, I used @keyframes gentle-float, @keyframes fade-in-up, and @keyframes card-appear. For the recommendation feature, I used selectors

like .recommendation-options:has(#mood-budget:checked) .result-budget to show the matching result.

Q8 – Accessibility & Usability

Question: Demonstrate keyboard navigation and visible focus on a key workflow.

I wanted people to be able to use HavenMade without always needing a mouse.

I added visible focus outlines so someone using a keyboard can tell which link or option is selected. I also made the recommendation choices actual HTML radio buttons instead of just clickable text.

I included descriptions for the food pictures using alt attributes, and I added CSS that turns off animations for people who prefer reduced motion.

I know accessibility is something I can keep improving, but I tried to include those features from the beginning.

Code reference: In css/style.css, a:focus-visible adds an outline to links, and .recommendation-options input:focus-visible + label highlights a focused recommendation option. The @media (prefers-reduced-motion: reduce) rule disables animations and transitions. Images in index.html, recipes.html, and recipe-detail.html have descriptive alt text.

Q9 – Kanban & Collaboration

Question: How did you limit work in progress and prioritize finishing tasks?

I used Trello to keep track of everything I needed to finish. My board has five columns: Backlog, To Do, In Progress, Review, and Done.

I worked on this project individually, so Trello helped me stay organized and see what I had already finished.

I focused on getting the main website pages working before spending more time on extra features. When something needed testing, I could keep it in Review instead of marking it done too early.

I think using Trello helped because I could actually see my progress instead of trying to remember everything.

Project reference: My Trello board is called **HavenMade / CSC 4370 Project 1**. It includes tasks for the website pages, styling, testing, documentation, and final submission.

Q10 – Testing & Debugging

Question: Describe a defect, the cause you traced, and the code change that fixed it.

One problem I ran into was that some of my recipe pictures weren't showing up after I uploaded the website to the CODD server.

The pictures worked in my local project, so I knew the HTML image paths weren't necessarily the problem. I checked the files on the server and found that their permissions needed to be changed.

After I updated the file permissions, I refreshed the published website and the pictures showed up.

This taught me that something can work on my computer but still have problems when it's uploaded to a server.

Technical reference: My images are stored in the images folder and linked using paths such as images/salmon.JPG and images/chickensoup.jpeg. I fixed the server permissions using chmod 644 for image files and chmod 755 for directories. The fix was made on CODD, not by changing my HTML or CSS.

Q11 – Extra Features & Scope

Question: How does your HTML/CSS UI simulate its AI concept without claiming real AI functionality?

My recipe recommendation feature is a simulation. It doesn't use real AI, and I didn't connect it to an AI service.

Instead, I used HTML radio buttons and CSS to create four possible recommendations. When someone selects a mood, the website displays the matching recipe, a short explanation, and a link to the cooking instructions.

I chose this approach because I wanted to add something interactive while still following the project's HTML and CSS requirements.

Code reference: In recipes.html, the recommendation options use <input type="radio"> elements. In css/style.css, the recommendation cards are hidden by default

with .recommendation-result { display: none; }. The :has() and :checked selectors display the correct result. The page also includes a note explaining that the recommendations are simulated.

Q12 – Mandatory Design Defense Questions

Q12.1 – Feature Integration

Question: Trace one complete feature from its HTML structure through CSS selectors and assets to the final browser result. Which files connect it to the rest of the site?

I'll use my recipe recommendation feature as an example.

The feature starts in recipes.html, where I created four radio buttons for different cooking moods. I also created a separate recommendation card for each choice.

In css/style.css, I used CSS selectors to control which card is visible. For example, when the budget option is selected, the CSS displays the budget recommendation and keeps the others hidden.

That recommendation links to Chicken Noodle Soup on recipe-detail.html. The recipe detail page then shows the food picture, ingredients, and cooking instructions.

So the HTML creates the feature, the CSS controls how it looks and responds, and the recipe links connect everything together.

Specific code:

- recipes.html: id="mood-budget" and class="recommendation-result result-budget"
- css/style.css: .recommendation-options:has(#mood-budget:checked) .result-budget
- recipe-detail.html: id="chicken-soup"
- Image: images/chickensoup.jpeg

Q12.2 – Bug Fix

Question: Show a bug you fixed, explain its root cause, and demonstrate or document the before-and-after result.

A bug I fixed was the missing food pictures on my published website.

At first, some images weren't loading on CODD, even though they appeared correctly in my local files.

I checked the uploaded files and found that the issue was related to file permissions. I updated the permissions so the server could read the image files.

After that, I checked the live website again and the images loaded correctly.

The main thing I learned was to always test the published version of a website instead of assuming everything will work just because it works locally.

Technical reference: The images are in HavenMade/images/. I used file permissions of 644 for image files and 755 for directories. This was a server configuration fix, so there wasn't a separate HTML or CSS code change.

Q12.3 – Design Trade-Off

Question: Compare two layout or styling approaches you considered. Why did you choose one, and what trade-off did you accept?

I thought about making HavenMade more decorative, but I also didn't want the design to get in the way of the recipes.

I ended up choosing a simple card-based layout with rounded corners, food pictures, and plenty of space between sections.

I liked this approach because visitors can look through the recipes without feeling overwhelmed. It also works better when the screen gets smaller.

The trade-off is that the website doesn't have as many complicated visual effects, but I think the simpler layout fits the purpose better.

Code reference: In css/style.css, .recipe-grid controls the recipe card layout, and .recipe-card controls the appearance of each card. The media queries change the number of columns depending on screen width.

Q12.4 – Edge Case & Accessibility

Question: Demonstrate a keyboard, reduced-motion, long-content, or narrow-viewport edge case. How did you detect and resolve it?

One thing I wanted to make sure of was that the recipe recommendation feature would still be usable on a small screen.

The feature has four choices, so I didn't want them to become too crowded on a phone.

I added responsive CSS so the options can change from four columns to two columns and then one column on very narrow screens.

I also included visible keyboard focus and reduced-motion support so people can use the feature more comfortably.

Code reference: In css/style.css, .recommendation-options uses four columns by default. At max-width: 700px, it changes to two columns, and at max-width: 380px, it changes to one. The input:focus-visible + label selector adds a visible outline, and prefers-reduced-motion: reduce disables animation.

Q12.5 – Refactoring & Future Improvements

Question: Identify one fragile HTML or CSS pattern, propose specific refactor steps, and explain how you would test for visual and accessibility regressions.

One part of my website I would improve in the future is the recommendation feature.

Right now, it works with four choices, but I think it could become harder to manage if I added a lot more recipes or categories.

I would start by organizing the related CSS rules more clearly and making sure the class names stay consistent. I would also look for repeated styling that could be combined.

After making changes, I would test all four recommendation choices again. I would also check the feature on desktop and mobile, make sure keyboard focus still works, and confirm that the correct recipe appears every time.

I would want to improve the code without accidentally changing how the website looks or works.

Code reference: The recommendation logic is in css/style.css, especially the selectors using :has() and :checked. The matching HTML is in the recommendation section of recipes.html.

Project Links

Published Website:

https://codd.cs.gsu.edu/~hfreeman11/HavenMade/

GitHub Repository:

https://github.com/Hfreeman1/HavenMade

Trello Board:

https://trello.com/b/qUbXhngy/havenmade-csc-4370-project-1

Additional Notes

HavenMade was built using HTML and CSS. The recipe recommendation feature is a CSS-only simulation and does not use JavaScript or an actual AI model.

The website includes four HTML pages, six recipes, responsive layouts, CSS animations, descriptive image text, keyboard focus styling, and reduced-motion support.

This document explains my project decisions and code as supplemental documentation.